

CSc 453: Fall 2025

Assignment 2 milestone 1

Start: Sept 19, 2025

Due: 11:59 PM, Sept 25, 2025

1. General

This milestone involves extending the scanner and parser from assignment 1 in two ways: first, extending the grammar handled by the parser to that for the [G1 subset of C--](#); and second, incorporating semantic checking into the parser. See **Supplemental material for the programming project**, under the **Contents** tab in D2L, for changes from the G0 subset of C-- to the G1 subset, as well as an example G1 program.

2. Required functionality

2.1. Programming requirements

Your program should implement a function `parse()` that extends your parser from Assignment 1 to parse the [G1 subset of C--](#) (the changes in G1 relative to G0 are shown [here](#)) and also, when specified via a command-line argument (see below), carry out semantic checking on G1. Specifically:

- If the program is invoked without the command-line argument `--chk_decl`, it should check whether the input program (read from **stdin**) is syntactically legal with respect to the grammar rules for the G1 language (specified in Section 2 of the [G1 language specification](#)).

The behavior in this case should be similar to that of the parser in Assignment 1 Milestone 2: if the input is syntactically correct, your parser should exit silently with exit status 0; if the input contains any syntax errors, your parser should print out an error message on **stderr** (see below) and exit with exit status 1. You are not required to implement any sort of error recovery.

- If the program is invoked with the command-line argument `--chk_decl`, your parser should additionally check the semantic rules for G1 (given in Section 3 of the [G1 language specification](#)).

The parser's behavior in this case should be as follows: in addition to checking the grammar rules (as it did before), it should also check the semantic rules given in Section 3 of the G1 language specification: if the input satisfies the semantic rules of G1, your parser should exit silently with exit status 0; if the input contains any errors, your parser should print out an error message on **stderr** (see below) and exit with exit status 1. You are not required to implement any sort of error recovery.

Example

Input	Program behavior	
	compile < input_file	compile --chk_decl < input_file
<code>int x; int f() { }</code>	ok	ok
<code>int x; int f() {</code>	error (unbalanced '{')	error (unbalanced '{')
<code>int x; int x; int f(){ }</code>	ok	error (x is doubly declared)

Driver code and program invocation

Your program should use the following driver file:

- [driver.c](#) -- a file containing the driver code for your parser.

This driver code has the following functionality: it scans the command-line arguments to see whether the program was invoked with the argument `--chk_decl`; if it was, it sets the global variable `chk_decl_flag` to 1, otherwise this global variable is set to 0. The driver then invokes your parser as in assignment 1.¹

The command-line argument `--chk_decl`, and thus the flag `chk_decl_flag`, controls whether or not the parser should perform semantic checking.

Makefile

You should submit a Makefile that provides (at least) the following targets:

- **make clean:**
Deletes any object files (`*.o`) as well as the file 'compile'
- **make compile:**
Compiles all the files from scratch and creates an executable file named 'compile'.

FIRST and FOLLOW sets

You can use the **gff** tool, which you used for Homework 1, to compute FIRST and FOLLOW sets for the grammar. Documentation on the use of gff is available [here](#). You can access a gff executable in either of two ways:

1. it is available as an executable on lectura at `/home/cs453/fall125/bin/gff`; or
2. you can grab the source code (available as a zip file [here](#)) and build the executable yourself.

Error messages

Your error messages should satisfy the following requirements:

- Each error message should contain the string **ERROR**. The grading code will use this to identify error messages.
- Each error message should contain a string **LINE *nnn***, where *nnn* is the line number where the error was detected. The grading code will use this to check whether errors are being detected at the right place.

See the specification for Assignment 1 Milestone 2 for common mistakes in dealing with syntax errors.

¹ You may notice that the driver file also mentions printing ASTs and code generation. This is so that we can use the same driver file for future assignments as well. For this assignment, you can simply ignore this.

The case (upper/lower/mixed) of the strings ERROR and LINE mentioned above is not important: I used upper case just to make the strings conspicuous.

Apart from the two requirements above, you are free to craft your error message as you see fit. You should try to give error messages that are helpful to the user—imagine you were using your compiler "for real" and consider whether your error messages would be helpful for the user to understand and locate the problem.

2.2. Interface requirements

Your program should be written to interface appropriately with the driver code mentioned above. To this end, your code should provide the following:

```
int parse(); /* the parser function */
```

The driver code will set the value of an int variable `chk_decl_flag` based on whether or not a command-line argument `--chk_decl` is specified (see the code in the file **driver.c** linked above). The parser should use the value of this variable to control semantic checking, such that semantic checking is performed if and only if `chk_decl_flag` has a non-zero value.

3. Files you need to submit

- Makefile that supports the functionality described above;
- `driver.c` -- the driver file provided to you for this assignment;
- any additional files implementing your scanner and parser.